MODULE II · DATA · WEEK 6

Automated Data Collection

Web scraping and APIs

M. Fuat Kına

WHERE DATA COMES FROM

Four routes into a dataset

01	**Ready-made archives**	World Bank, OECD, TÜİK, GLOCON, ICPSR. Cheapest route — but somebody else fixed the unit of analysis for you.
02	**Application programming interfaces**	YouTube, Reddit, Wikipedia, Crossref, central banks. A documented, machine-readable door the platform opened on purpose.
03	**Web scraping**	No door, so you read the page as a browser would. Fragile, slower, and the only option when there is no API.
04	**Model-assisted extraction**	Turning unstructured text, images or PDFs into variables. Module IV takes this seriously.

Ask which route your question needs before you write a line of code. The cheapest adequate one wins.

CHOOSING BETWEEN THEM

An API if you can, a scraper if you must

ASK THE PLATFORM

API

- Structured JSON — no parsing guesswork
- Documented fields with stable names
- Survives a redesign of the website
- Explicit quotas you can plan around
- But: only what the platform chooses to expose

READ THE PAGE

Scraper

- Works on any page a browser can render
- You define the fields yourself
- Breaks the moment a class name changes
- Rate limiting and blocking are on you
- But: often the only access that exists

Same rule either way: collect the minimum you need, and store it the moment you have it.

WEB SCRAPING

Five steps, every time

1

Inspect

Open developer tools. Find the element that holds what you want.

```
right-click → Inspect
```

→

2

Request

Fetch the raw HTML the server sends back.

```
requests.get(url)
```

→

3

Parse

Turn that string into a navigable tree.

```
Selector(text=html)
```

→

4

Extract

Address the nodes you want with XPath or a CSS locator.

```
sel.xpath(path)
```

→

5

Store

Clean, put it in a DataFrame, write it to disk.

```
df.to_csv(...)
```

Libraries: Scrapy, BeautifulSoup, requests, lxml, Selenium (for pages built by JavaScript)

The first step is the one students skip — and the one that decides whether the other four work.

THE RAW MATERIAL

URLs and HTML

A URL is a recipe
path + "/" + searchterm + "?" + parameters

Change one parameter and you have a new query. That is what turns one page into a loop.

An HTML document is a tree
elements + attributes + text, related as parents, children, siblings and descendants.

```
https://www.aljazeera.com/search/Turkey?sort=date

  path ──────────┘  term ─┘   parameters ─┘
```

```
<html>
  <body>
    <div id="div1" class="class-1">
      <p class="class-1 class-2">Hello World!</p>
      <p>Data Science!</p>
    </div>
  </body>
</html>
```

id is unique on a page; class is not. That single fact decides most of your selectors.

VOCABULARY

The elements you will actually meet

METADATA

<head> collection of metadata
<title> title of the document

GROUPED CONTENT

** ** container
<p> paragraph

LISTS

<ul> unordered list
<ol> ordered list
<li> entry of a list

SECTIONS

<body> main contents
<section> section of the document
<h1> <h2> headlines

LINKS

<a> hyperlink; refers via the href attribute

TABLES

<table> table
<tr> row of a table
<td> cell of a table

You do not need to write HTML. You need to read it well enough to point at one node.

ADDRESSING A NODE

XPath: the grammar

`/`	move forward one generation
`//`	look forward to all future generations
`[ ]`	specify the order, or a condition
`@`	address an attribute rather than an element
`*`	any element (wildcard)
`text()`	the text carried by the element

```
'/html/body/div[2]//table'

every table anywhere below the second
div, which is a child of body.
```

```
'//p'
'/head/body//div'
'/div[3]//p'

'//span[@class="span-class"]'
all span elements whose class is span-class
```

Credit for the XPath and CSS material: DataCamp, "Web Scraping with Python".

ADDRESSING A NODE

XPath: wildcards, attributes, contains()

`'/html/body/*'`	every child of the body element
`'/html/body//*'`	every descendant of the body element
`'//*'`	every element in the document
`'//div[@id="uid"]/a/@href'`	the href value of a link inside that div
`'//a[contains(@class,"snippet")]'`	class attributes hold several names — match a part

Prefer a short, attribute-anchored path over the long absolute one your browser copies for you.

EXERCISE

Write four XPaths for this document

```
<html>
  <body>
    <div id="div1" class="class-1">
      <p class="class-1 class-2">Hello World!</p>
      <div id="div2">
        <p id="p2" class="class-2">Choose
          <a href="http://datascience.com">
             Data Science!</a>!
        </p>
      </div>
    </div>
    <div id="div3" class="class-2">
      <p class="class-2">Thanks for Watching!</p>
    </div>
  </body>
</html>
```

01

Select the paragraph carrying “Thanks for Watching!”, using its class attribute.

02

Select the paragraph carrying “Hello World!”, using its class attribute.

03

Select the href value of the Data Science hyperlink — the attribute, not the element text.

04

Select every href of the a elements whose class attribute contains “package-snippet”.

Two of these have more than one correct answer. Say out loud which one you would trust in six months.

THE OTHER SYNTAX

CSS locators

Find by class with a period

`p.class-1 → every paragraph belonging to class-1`

Find by id with a hash

`div#uid → the div whose id is uid`

Shorter to write, harder to express conditions in. Scrapy accepts both, and you can mix them within one spider.

```
XPATH
xpath = '/html/body//div/p[2]'

CSS
css = 'html > body div > p:nth-of-type(2)'
```

```
paragraphs inside div#uid that are class1
css_locator = 'div#uid > p.class1'
everything belonging to class1
css_locator = '.class1'
```

```
attributes
xpath = '//div[@id="uid"]/a/@href'
css   = 'div#uid > a::attr(href)'
```

Pick one and be consistent inside a project; a spider that mixes styles is harder to debug.

EXTRACTING TEXT

One slash or two — the difference that bites

```
<p id="p-example">
  Hello world!
  Try <a href="http://www.datacamp.com">DataCamp</a> today!
</p>
```

XPath — text()

```
sel.xpath('//p[@id="p-example"]/text()')
['\n Hello world!\n Try ', ' today!\n']

sel.xpath('//p[@id="p-example"]//text()')
['\n Hello world!\n Try ', 'DataCamp',
 ' today!\n']
```

CSS — ::text

```
sel.css('p#p-example::text')
['\n Hello world!\n Try ', ' today!\n']

sel.css('p#p-example ::text')
['\n Hello world!\n Try ', 'DataCamp',
 ' today!\n']
```

A single slash reads only the element's own text. A double slash reaches into its children — including the link.

SCALING UP

Scraping with a Spider

A Selector handles one page you already downloaded. A Spider handles a site: it queues requests, follows links, retries failures and respects a delay between calls.

`start_requests()`
where the crawl begins.

`parse()`
what to do with each response — extract, or yield another request.

```
import scrapy

class DCspider(scrapy.Spider):
    name = "dcspider"

    def start_requests(self):
        urls = ['https://www.datacamp.com/courses/all']
        for url in urls:
            yield scrapy.Request(url=url,
                                 callback=self.parse)

    def parse(self, response):
        links = response.css(
            'div.course-block > a::attr(href)').extract()
        for link in links:
            yield response.follow(url=link,
                                  callback=self.parse2)

    def parse2(self, response):
        # parse the course pages here
```

yield, not return: a spider hands work back to the scheduler and carries on.

CASE STUDY

Real estate listings, one district at a time

01 **Find the pattern in the URL** — district and price bounds are ordinary query parameters — so they are ordinary loop variables.

02 **Extract titles, links, prices** — one XPath each, taken from the listing card and checked against a second card.

03 **Clean what comes back** — strip residual tags, drop the empty strings the grid layout leaves behind.

04 **Follow the links** — the listing page has the building age; the results page never did.

05 **Assemble the DataFrame** — and check that your three lists are still the same length. They often are not.

Every XPath here is a hostage to a redesign. Save the raw HTML alongside the parsed table, and date your files.

PART TWO

Application Programming Interfaces

When the platform hands you the data, the hard part stops being extraction and starts being sampling.

HOW AN API WORKS

Anatomy of a request

Endpoint

The address of one resource. /search is a different endpoint from /videos, with different fields.

Parameters

Your query, as key–value pairs: what to look for, how many, in which order, from when.

Authentication

An API key or an OAuth token identifies you. It is a secret — it does not belong in your repository.

Response

JSON: nested dictionaries and lists. json.loads() gives you Python objects; pandas gives you a table.

```
import requests

endpoint = 'https://www.googleapis.com'
           '/youtube/v3/search'

params = {
    'part': 'snippet',
    'q': 'deprem',
    'type': 'video',
    'maxResults': 50,
    'key': API_KEY,
}

r = requests.get(endpoint, params=params)
data = r.json()

r.status_code  # 200 is the only
               # one you want
```

Read the documentation before the tutorial. Field names change; blog posts do not.

WHAT WILL GO WRONG

Quotas, pages and failures

Quotas

Requests are rationed, and not equally. On the YouTube Data API a search costs 100 units of a 10,000-unit daily budget; reading a video's details costs 1. Design the cheap query.

Pagination

You are handed a page at a time, plus a nextPageToken. Loop until the token disappears — and cap the loop, or you will discover the quota the hard way.

Failures

403 means quota or permission, 404 a bad endpoint, 429 too fast. Catch them, back off, retry — and write results to disk as you go, not at the end.

Sampling

The deepest problem is not technical. An API returns what its ranking returns; “relevance” is not a random sample, and a deleted comment is silently absent.

Olteanu et al. (2019) and Sen et al. (2021) are about the fourth card. Read them before you trust your N.

USE CASE

Three endpoints, one dataset

search `.list()`	query → up to 50 video ids per page	**100 units**
↓		
videos `.list()`	ids → title, channel, publish date, view / like / comment counts	**1 unit**
↓		
commentThreads `.list()`	video id → top-level comments and their authors	**1 unit**

A researchable table: one row per comment, with the video and channel attributes attached. From there it is sentiment, topic models, or a time series of attention.

BEFORE YOU PRESS RUN

Because you can is not because you may

robots.txt	Read site.com/robots.txt. It is a request, not a lock — but ignoring it is a decision you have to defend.
Terms of service	Many platforms forbid scraping outright while running an API. That is usually the answer to which route to take.
Rate	One request per second, a real User-Agent, and a cache. A scraper that hurts a server is a bad scraper.
Persons	Public does not mean consented. Usernames, locations and timestamps re-identify people; store the minimum and aggregate early.
Ethics review	If your unit of analysis is a person rather than a page, your IRB wants to hear about it in advance.

Reading: Jünger (2018) on collection logics; Schäfer & Van Es (2017), ch. 10, on social data APIs.

ASSIGNMENT 3 · RELEASED TODAY

Web scraping and APIs

Collect

Build one dataset twice over: a scraped table and an API-based table. They may be on different topics — the point is to write both kinds of collector.

Document

A notebook with comments a reader can follow, and a short note on what you sampled, what you missed, and why.

Deposit

Push the notebook and the resulting CSV to your Python_Course repo. Never push your API key.

Next week — Machine learning for social research begins. Bring the midterm report.